

LAPORAN MODUL 14 PRAKTIKUM ALGORITMA DAN PEMROGRAMAN LANJUTAN



Disusun oleh :

Nama : Irvan Cahya Nugraha
NIM : 245410034
Kelas : Informatika 1

**PROGRAM STUDI INFORMATIKA
PROGRAM SARJANA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS TEKNOLOGI DIGITAL INDONESIA
YOGYAKARTA
2024 / 2025**

MODUL 14

SEARCHING

A. TUJUAN PRAKTIKUM

1. Mahasiswa dapat melakukan pencarian data dengan metode linear search dan mengimplementasikannya dalam program

B. DASAR TEORI

SEARCHING (PENCARIAN)

Searching (pencarian) adalah aplikasi komputer yang sangat penting. Dalam pencarian, hal yang paling penting adalah adanya kunci pencarian. Searching adalah algoritma untuk mencari data yang berada pada suatu kumpulan data tertentu. Pencarian terhadap data, bisa merupakan data yang sudah terurut maupun yang belum terurut. **Pencarian Linear** Pencarian yang membandingkan data kunci dengan seluruh data, mulai dari data pertama kasus terburuk adalah data yang dicari ditemukan pada deret terakhir.

Contoh :

Larik ke	:	0	1	2	3	4	5
Data	:	10	4	14	7	5	23

Akan dicari data 14. Langkah yang ditempuh adalah mulai dari data ke-0 dibandingkan dengan data yang dicari

Larik ke-0 : 10 tidak sama dengan 14, pencarian bergeser ke larik ke-1

Larik ke-1 : 4 tidak sama dengan 14, bergeser lagi ke larik ke-2

Larik ke-2 : 14 sama dengan 14, pencarian berakhir karena data ditemukan.

C. PEMBAHASAN LISTING PRAKTIK

Praktik 1

```
import java.util.Scanner;
public class Larik9
{
    public static void main(String args[])
    {
        Scanner input = new Scanner(System.in);
        int i, x;
        boolean ketemu;
        int data[] = {12, 20, 14, 9, 34};
        System.out.print("Masukan Data yang dicari = ");
        x = input.nextInt();
        ketemu = false;
        for(i = 0; i < 5; i++)
        {
            if (data[i] == x)
            {
                ketemu = ! ketemu;
                break;
            }
        }
        if (ketemu)
            System.out.println("Data tersebut ada pada posisi ke-"
                +(i+1));
        else
            System.out.println("Data tidak ketemu !");
    }
}
```

C:\WINDOWS\system32\cmd. x + v

Masukan Data yang dicari = 12
Data tersebut ada pada posisi ke-1
Press any key to continue . . . |

Pembahasan : Program di atas adalah program pencarian linear menggunakan for dan if. Dalam program ini, variabel data adalah array integer yang berisi 5 elemen: {12, 20, 14, 9, 34}. Pengguna diminta memasukkan angka melalui Scanner input, lalu disimpan ke dalam variabel x. Pencarian dilakukan dengan perulangan for, indeks i berjalan dari 0 hingga 4. Pada setiap iterasi, data[i] dibandingkan dengan x, dan jika ditemukan sama, maka variabel boolean ketemu diubah menjadi true melalui operasi ! ketemu, lalu break. Jika pencarian menemukan data yang sama, maka akan ditampilkan posisi i+1 ke layar, karena posisi array dimulai dari 0. Jika tidak ditemukan, maka akan mencetak “Data tidak ketemu!”.

LATIHAN

```
import java.util.Scanner;
public class LinearSearchR
{
    static int data[]={12, 20, 14, 9, 34};

    public static int linearSearch (int kunci, int indeksAwal)
    {
        if(kunci == data[indeksAwal])
            return indeksAwal;
        if(indeksAwal + 1 < data.length)
            return linearSearch(kunci, indeksAwal + 1);
        return -1;
    }

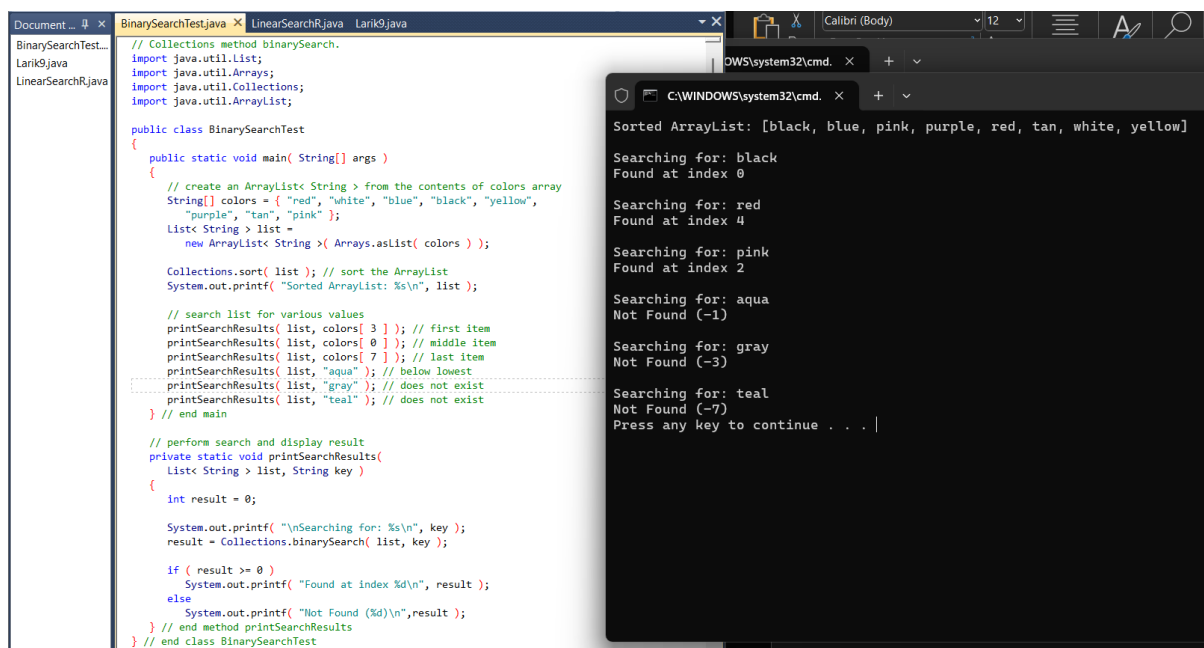
    public static void main(String args[])
    {
        Scanner input = new Scanner(System.in);
        int i, x;
        System.out.print("Masukan Data yang dicari = ");
        x = input.nextInt();
        i = linearSearch(x,0);
        if (i!=-1)
            System.out.println("Data tersebut ada pada posisi ke-"+i);
        else
            System.out.println("Data tidak ketemu !");
    }
}
```

C:\WINDOWS\system32\cmd. x + v

Masukan Data yang dicari = 12
Data tersebut ada pada posisi ke-0
Press any key to continue . . . |

Pembahasan : : Program latihan ini adalah versi modifikasi dari praktik sebelumnya, menggunakan pendekatan rekursif. Array data tetap berisi {12, 20, 14, 9, 34}. Fungsi `linearSearch(int kunci, int indeksAwal)` akan mengecek apakah elemen di `data[indeksAwal]` sama dengan kunci. Jika sama, maka `indeksAwal` dikembalikan sebagai hasil. Jika belum ditemukan dan indeks belum mencapai batas array, maka fungsi memanggil dirinya sendiri dengan `indeksAwal + 1`, inilah bagian rekursifnya. Proses ini akan berhenti ketika kunci ditemukan atau indeks melebihi batas array, yang menyebabkan -1 dikembalikan. Di `main()`, nilai `i` yang dikembalikan dari `linearSearch()` menentukan apakah data ditemukan atau tidak. Program ini menunjukkan cara kerja pemanggilan rekursif yang menggantikan loop.

D. PEMBAHASAN TUGAS



The image shows a screenshot of an IDE with two windows. The left window displays the source code for `BinarySearchTest.java`, and the right window shows the command prompt output.

```

// Collections method binarySearch.
import java.util.List;
import java.util.Arrays;
import java.util.Collections;
import java.util.ArrayList;

public class BinarySearchTest
{
    public static void main( String[] args )
    {
        // create an ArrayList< String > from the contents of colors array
        String[] colors = { "red", "white", "blue", "black", "yellow",
            "purple", "tan", "pink" };
        List< String > list =
            new ArrayList< String >( Arrays.asList( colors ) );

        Collections.sort( list ); // sort the ArrayList
        System.out.printf( "Sorted ArrayList: %s\n", list );

        // search list for various values
        printSearchResults( list, colors[ 3 ] ); // first item
        printSearchResults( list, colors[ 0 ] ); // middle item
        printSearchResults( list, colors[ 7 ] ); // last item
        printSearchResults( list, "aqua" ); // below lowest
        printSearchResults( list, "gray" ); // does not exist
        printSearchResults( list, "teal" ); // does not exist
    } // end main

    // perform search and display result
    private static void printSearchResults(
        List< String > list, String key )
    {
        int result = 0;

        System.out.printf( "\nSearching for: %s\n", key );
        result = Collections.binarySearch( list, key );

        if ( result >= 0 )
            System.out.printf( "Found at index %d\n", result );
        else
            System.out.printf( "Not Found (%d)\n", result );
    } // end method printSearchResults
} // end class BinarySearchTest
  
```

```

C:\WINDOWS\system32\cmd. x
+ v

Sorted ArrayList: [black, blue, pink, purple, red, tan, white, yellow]

Searching for: black
Found at index 0

Searching for: red
Found at index 4

Searching for: pink
Found at index 2

Searching for: aqua
Not Found (-1)

Searching for: gray
Not Found (-3)

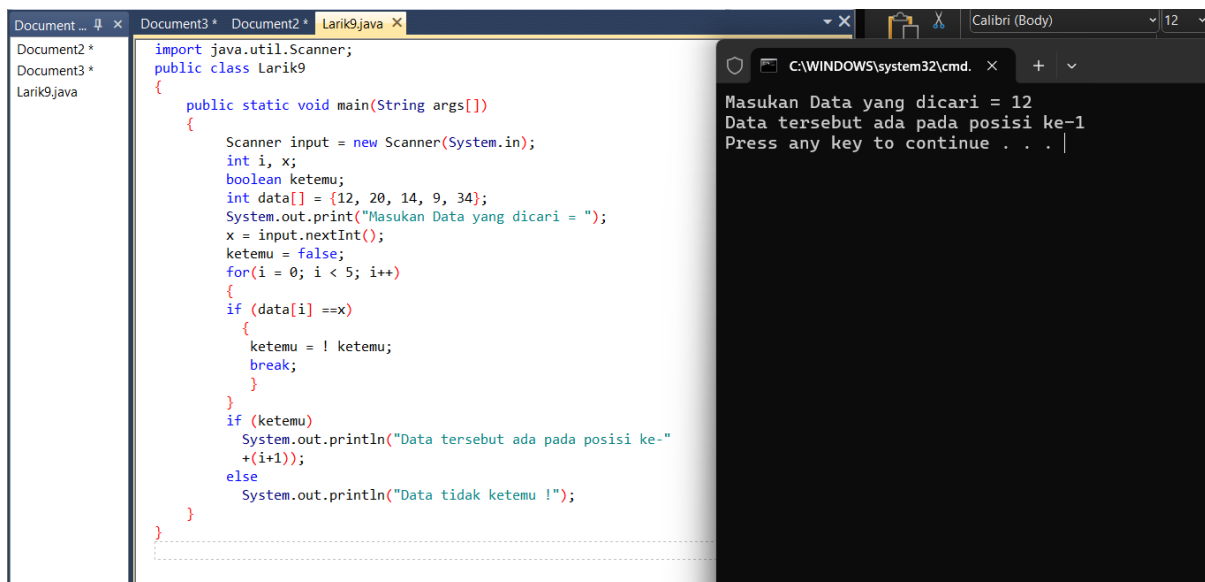
Searching for: teal
Not Found (-7)
Press any key to continue . . . |
  
```

Pembahasan : Binary Search `Collections.binarySearch`: Pada tugas ini digunakan metode pencarian biner (binary search) melalui method `Collections.binarySearch()` terhadap `ArrayList` bertipe `String`. Sebelum pencarian dilakukan, array `colors` diubah menjadi `List` dengan `Arrays.asList()`, lalu disortir terlebih dahulu dengan `Collections.sort()` agar bisa diterapkan binary search. Fungsi `printSearchResults()` akan mencetak hasil pencarian, apakah ditemukan (`result >= 0`) atau tidak (`result < 0`). Proses pencarian dilakukan terhadap beberapa nilai seperti `colors[3]`, `colors[0]`, `colors[7]`, serta nilai-nilai yang tidak ada dalam list seperti "aqua", "gray", dan "teal". Binary search jauh lebih cepat dibanding linear search karena tidak mengecek satu per satu, melainkan membagi list secara logika (divide and conquer).

E. KESIMPULAN

Metode pencarian linear mengecek elemen satu per satu dari awal sampai akhir array, cocok untuk data kecil atau tidak terurut. Versi rekursif dari linear search meniru perilaku perulangan melalui pemanggilan fungsi itu sendiri, tetapi secara teknis memakan stack call lebih dalam. Sementara itu, metode binary search bekerja lebih cepat karena membagi data yang sudah terurut untuk mempercepat proses pencarian, dan Java sudah menyediakan implementasi efisien melalui `Collections.binarySearch`. Dalam implementasi praktikum, memahami posisi indeks, logika perbandingan, serta syarat data terurut sangat penting dalam membedakan penggunaan linear dan binary search.

F. LAMPIRAN



The screenshot displays a Java IDE with a file named `Larik9.java` open. The code implements a linear search algorithm. It imports `java.util.Scanner`, defines a `main` method, and uses a `Scanner` to take user input. A boolean variable `ketemu` tracks if the data is found. A `for` loop iterates through an array `data` containing `{12, 20, 14, 9, 34}`. If the input matches an element, `ketemu` is set to `true` and the loop breaks. After the loop, the program prints the position of the found data or a message indicating it was not found.

```
import java.util.Scanner;
public class Larik9
{
    public static void main(String args[])
    {
        Scanner input = new Scanner(System.in);
        int i, x;
        boolean ketemu;
        int data[] = {12, 20, 14, 9, 34};
        System.out.print("Masukan Data yang dicari = ");
        x = input.nextInt();
        ketemu = false;
        for(i = 0; i < 5; i++)
        {
            if (data[i] ==x)
            {
                ketemu = ! ketemu;
                break;
            }
        }
        if (ketemu)
            System.out.println("Data tersebut ada pada posisi ke-"
                               +(i+1));
        else
            System.out.println("Data tidak ketemu !");
    }
}
```

To the right of the IDE, a command prompt window shows the program's execution. It prompts for the data to be searched (12) and outputs that it was found at position 1, followed by a prompt to press any key to continue.

```
C:\WINDOWS\system32\cmd. x + v
Masukan Data yang dicari = 12
Data tersebut ada pada posisi ke-1
Press any key to continue . . . |
```

